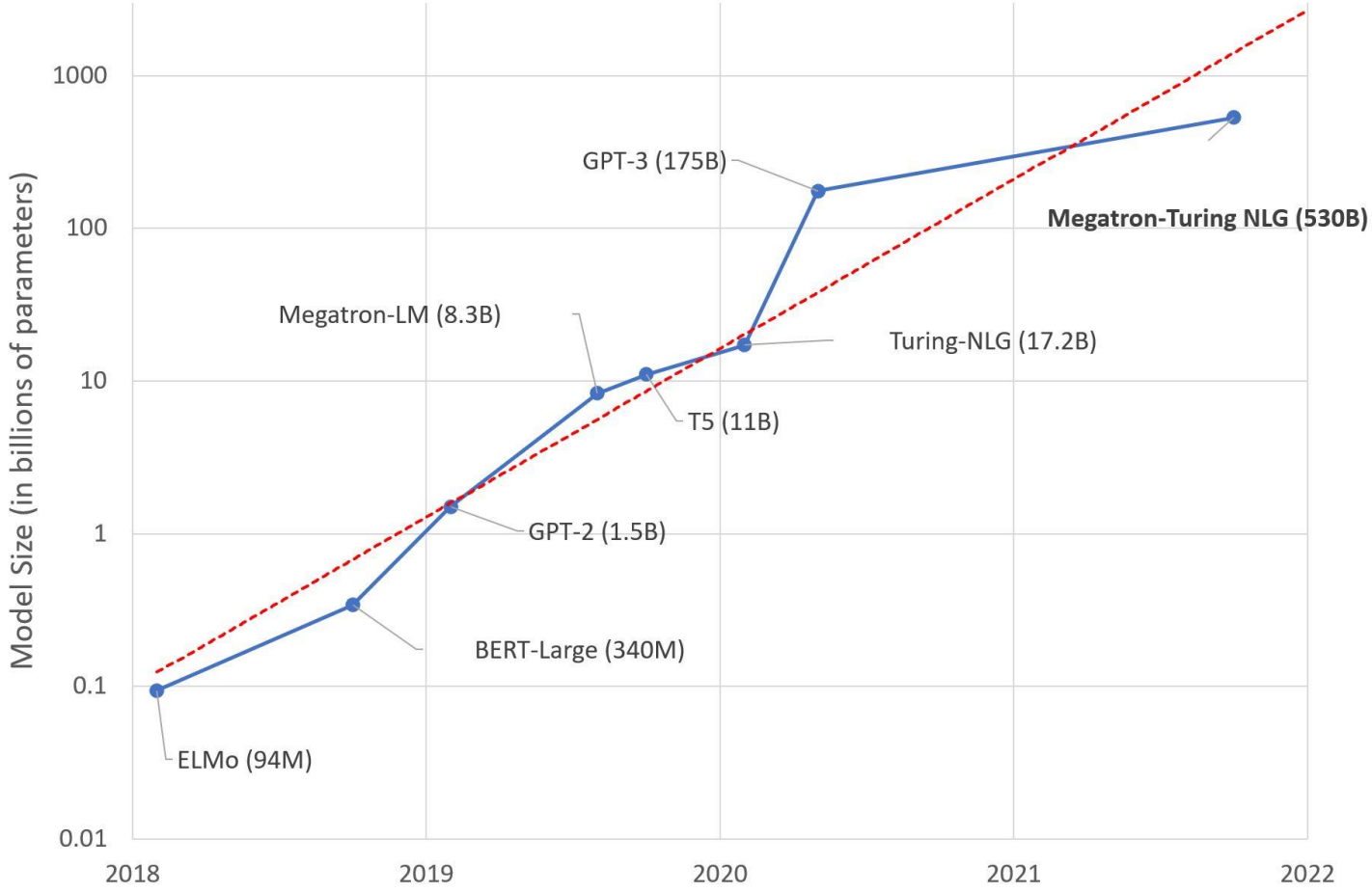
02 Deep Neural Networks

Scaling in deep learning



https://huggingface.co/blog/large-language-models

Universal Approximation Theorem

"If the network is large enough, it can model non-linear relationships of any arbitrary complexity"

Yes, but how can we train this model?

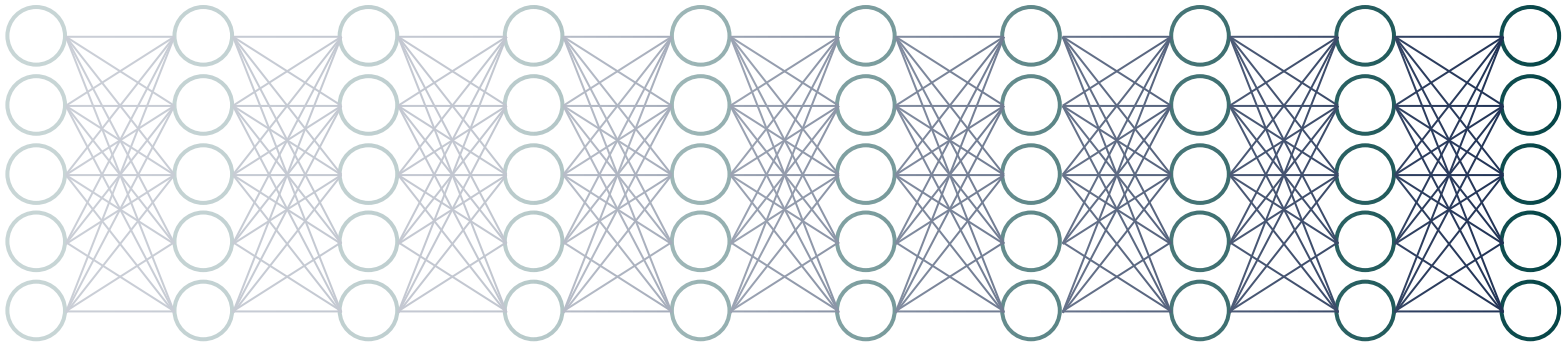
Vanishing/exploding gradients

Optimizers

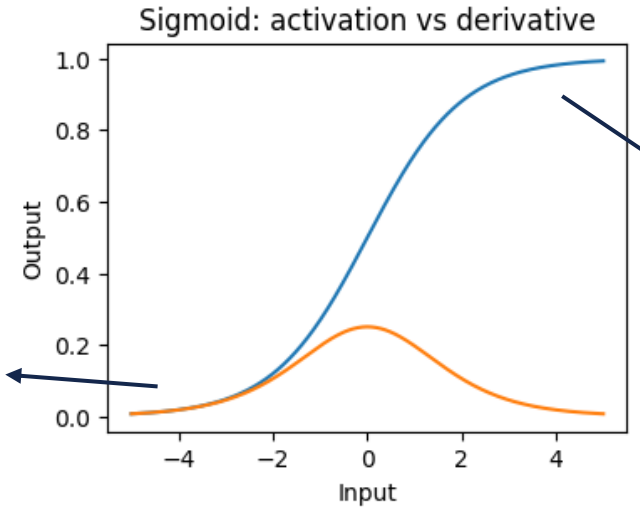
Regularization

GPUs = matrix operations on fire

Vanishing/exploding gradients



Backpropagation



Saturation

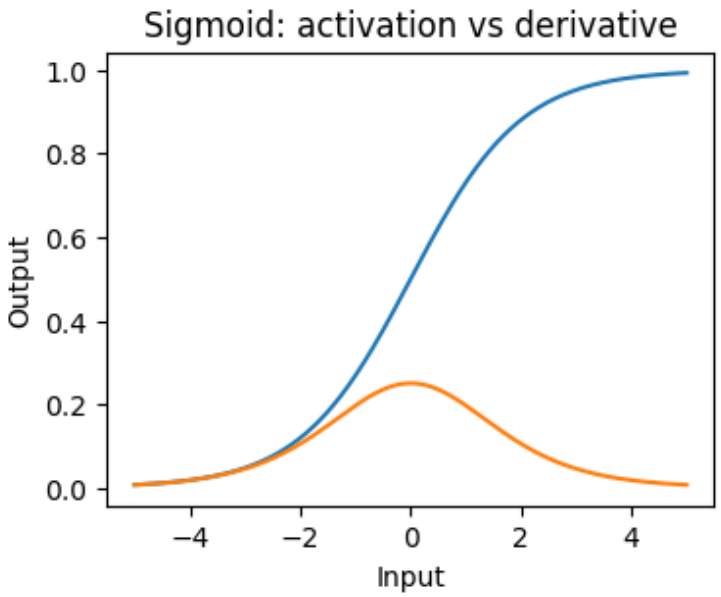
Saturation

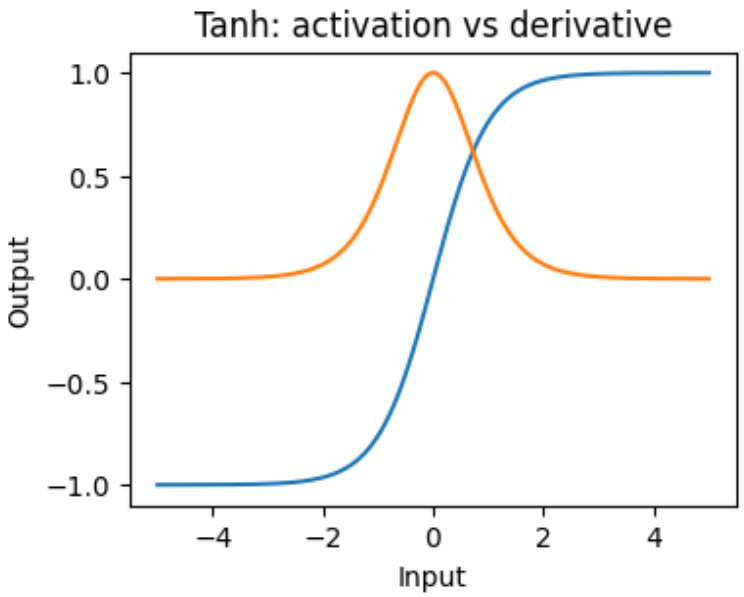
Multiplication of gradients

< 1 → exponentially smaller gradients

> 1 → exponentially larger gradients

Activation functions

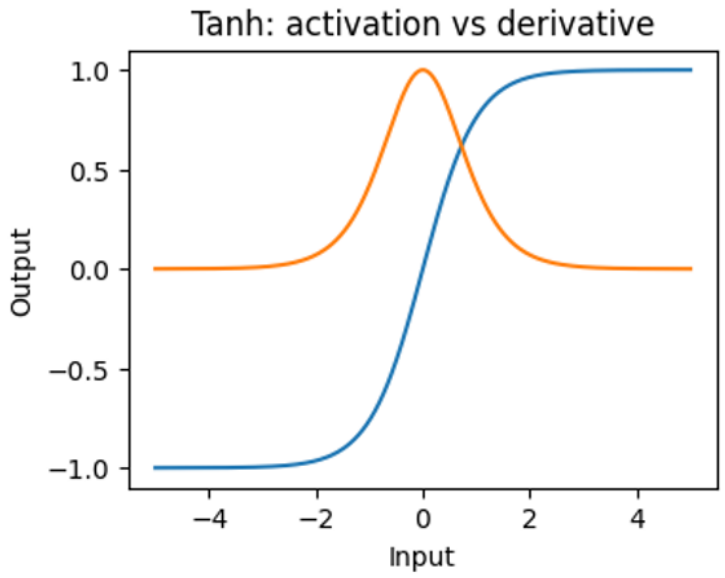


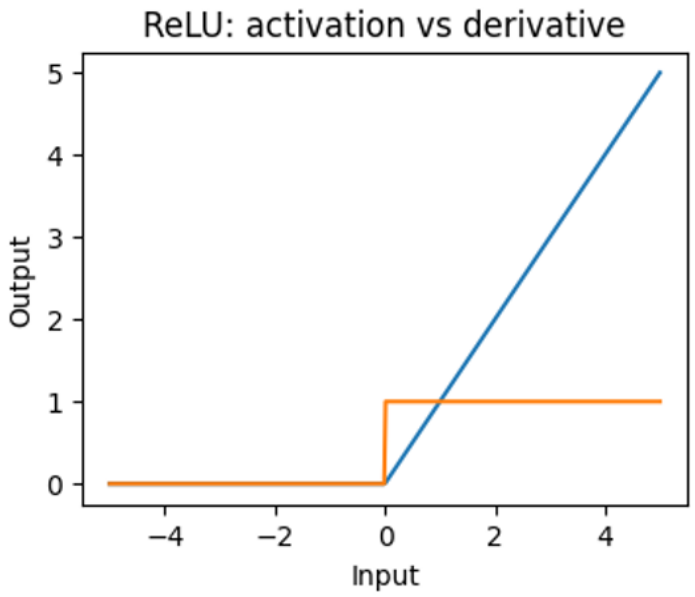


- Vanishing gradient problem
- Not centered around zero

- Zero-centered
- Steeper gradient (fast convergence)
- Vanishing gradient problem

Activation functions

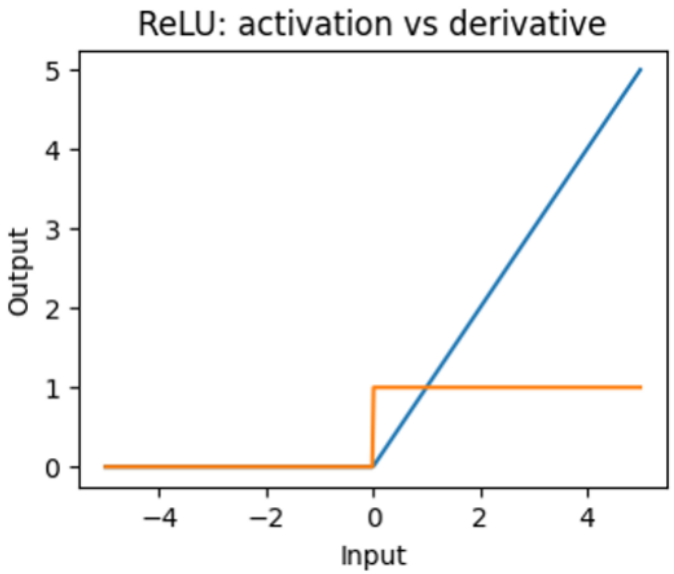


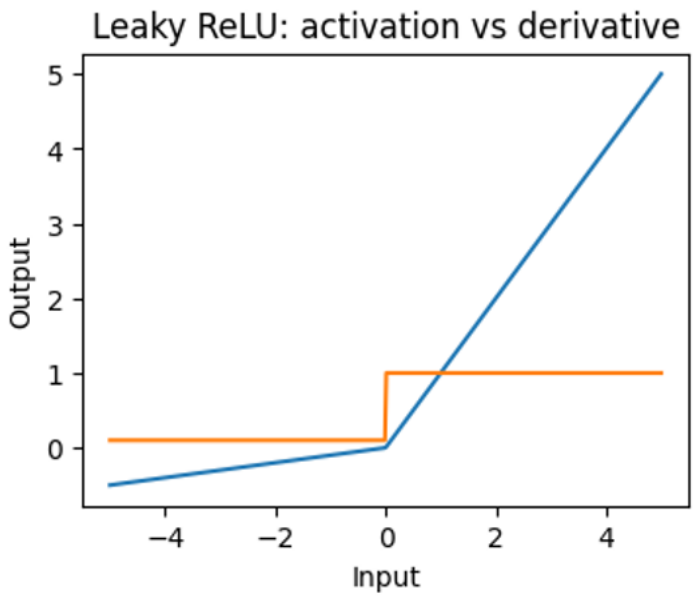


- Zero-centered
- Steeper gradient (fast convergence)
- Vanishing gradient problem

- No vanishing gradient
- Fast convergence
- Efficient computation (sparse activation)
- Dying neurons

Activation functions

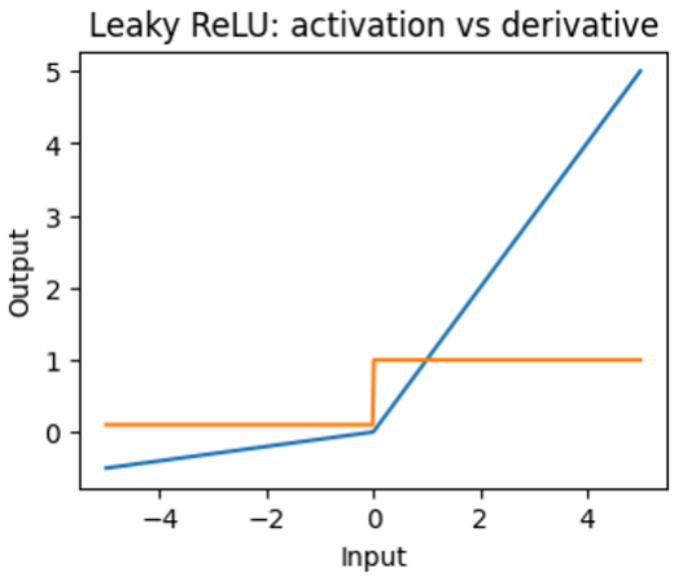


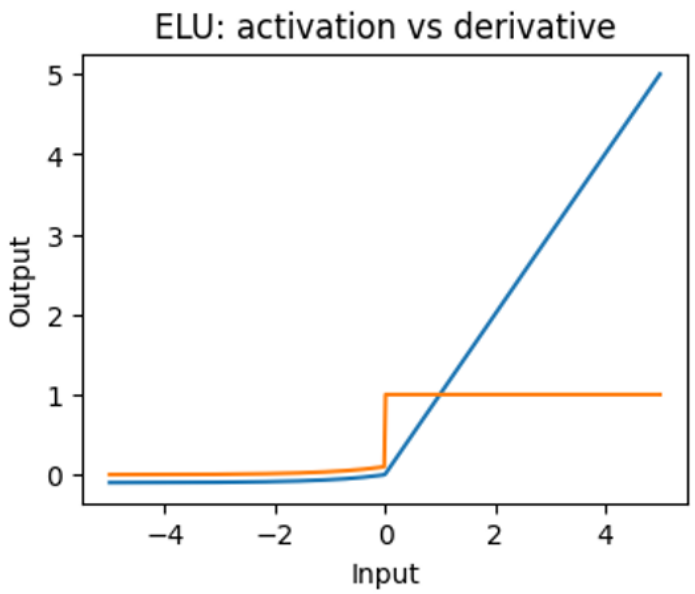


- No vanishing gradient
- Fast convergence
- Efficient computation (sparse activation)
- Dying neurons

- No vanishing gradient
- Fast convergence
- Efficient computation (sparse activation)
- No dying neurons
- Slope has to be optimized

Activation functions





- No vanishing gradient
- Fast convergence
- Efficient computation (sparse activation)
- No dying neurons
- Slope has to be optimized

- Fast convergence
- Efficient computation (sparse activation)
- No dying neurons
- Risk of saturation

Activation functions

- ReLU: **default choice** for most neural networks (simplicity and efficiency)
- ELU: for tasks requiring **faster convergence** or where **negative inputs** hold valuable information
- Sigmoid/Tanh: recurrent neural networks (RNNs, **temporal dynamics**)

Vanishing/exploding gradients

Optimizers

Regularization

GPUs = matrix operations on fire

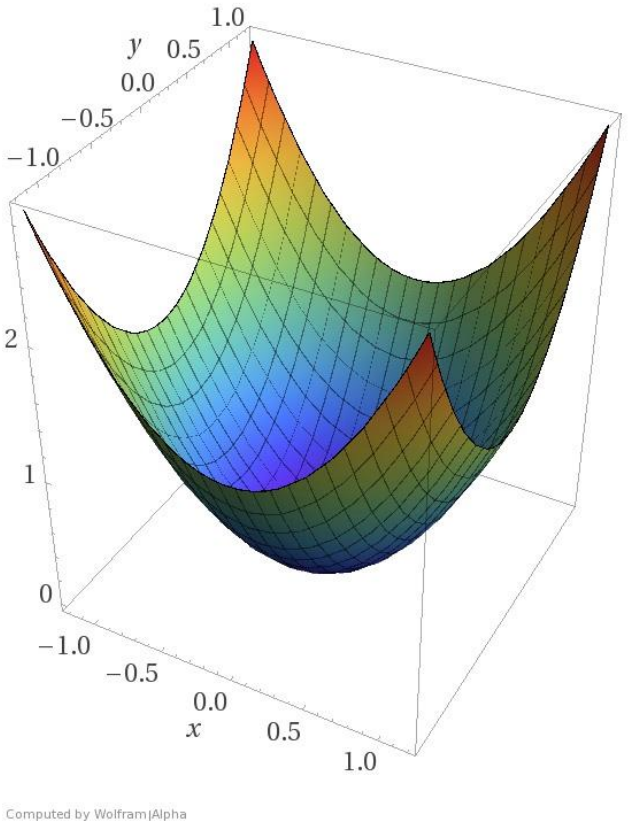
Optimizers

Deeper networks (and more complex data
→ training becomes challenging

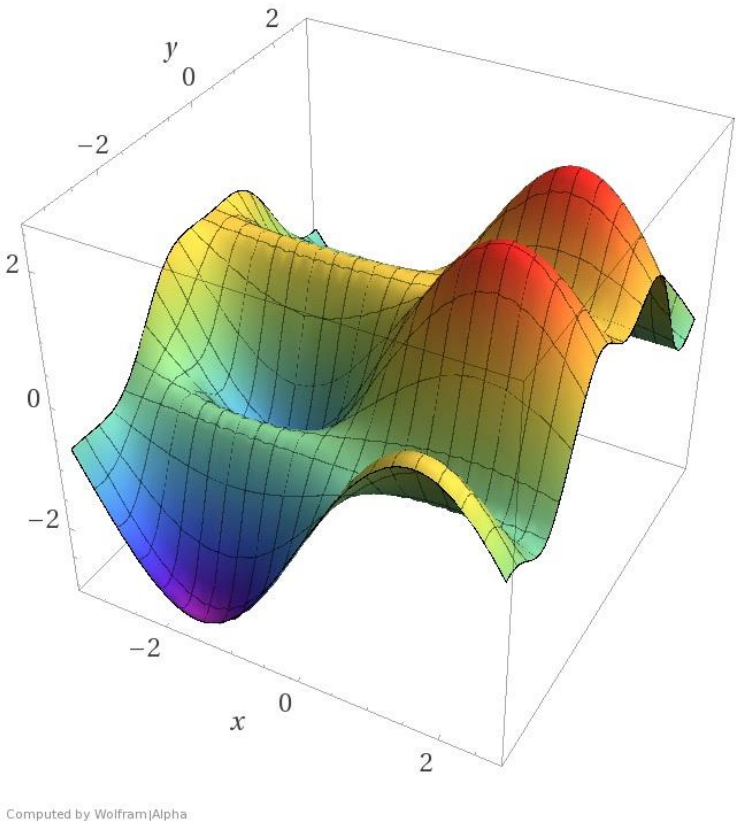
Loss surface
- highly non-convex
- with lots of local minima

→ Need to optimize the architecture and our training approach

Logistic regression



Simple neural network



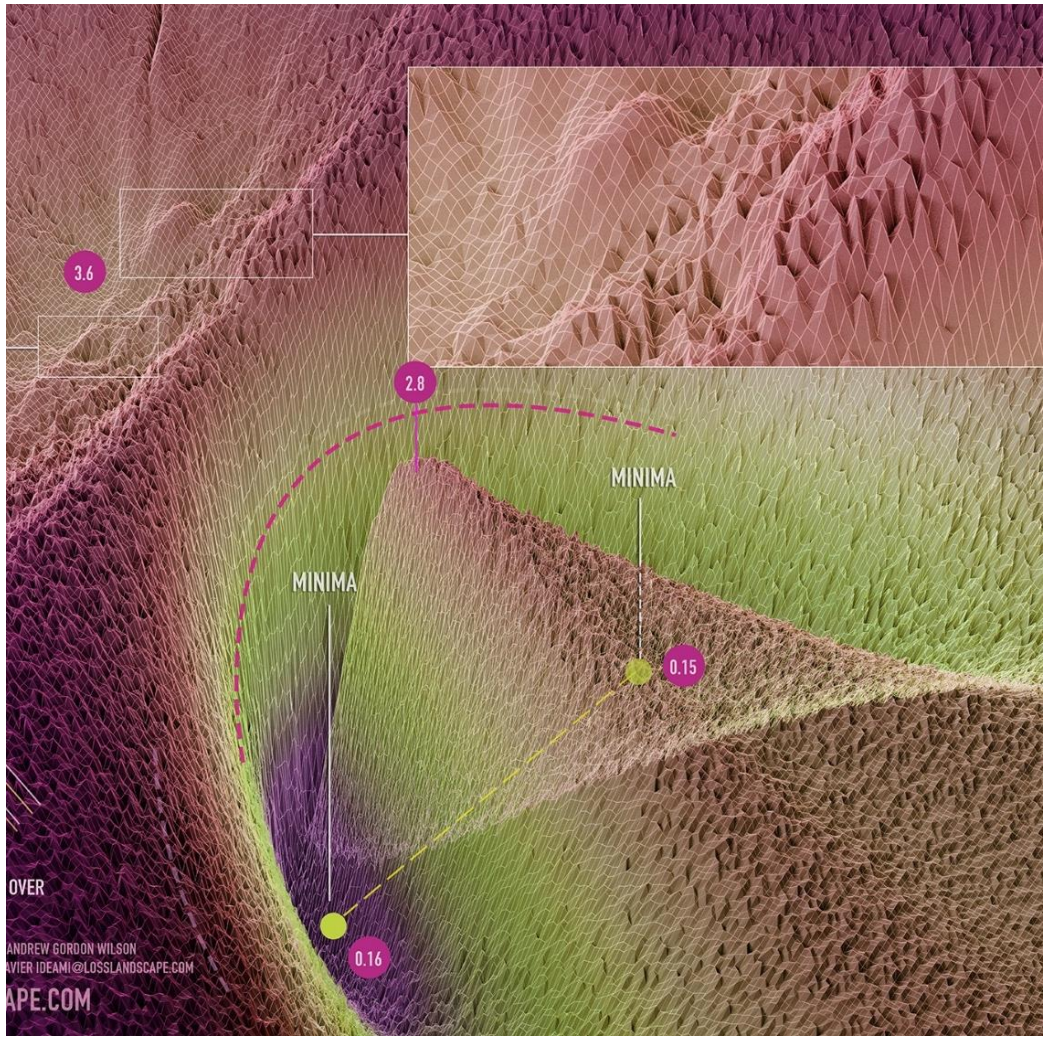
Optimizers

Deeper networks (and more complex data
→ training becomes challenging

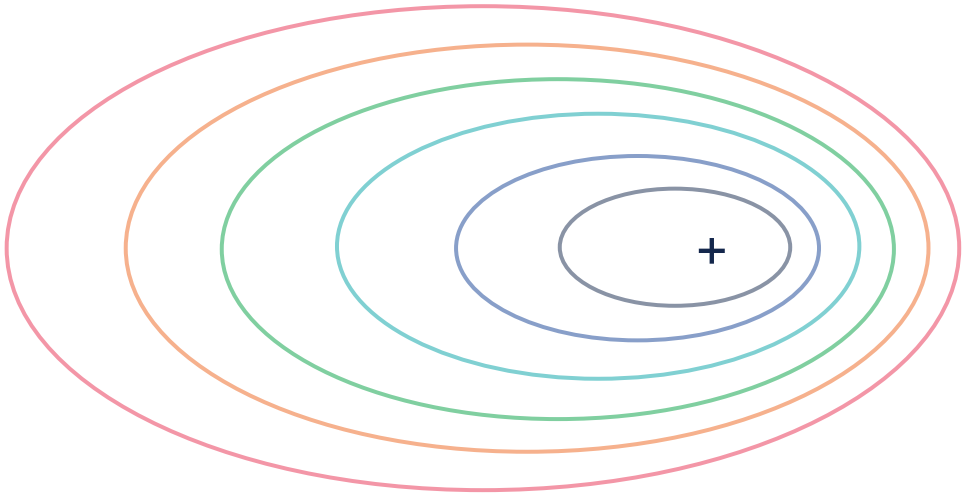
Loss surface
- highly non-convex
- with lots of local minima

→ **Need to optimize the architecture and our training approach**

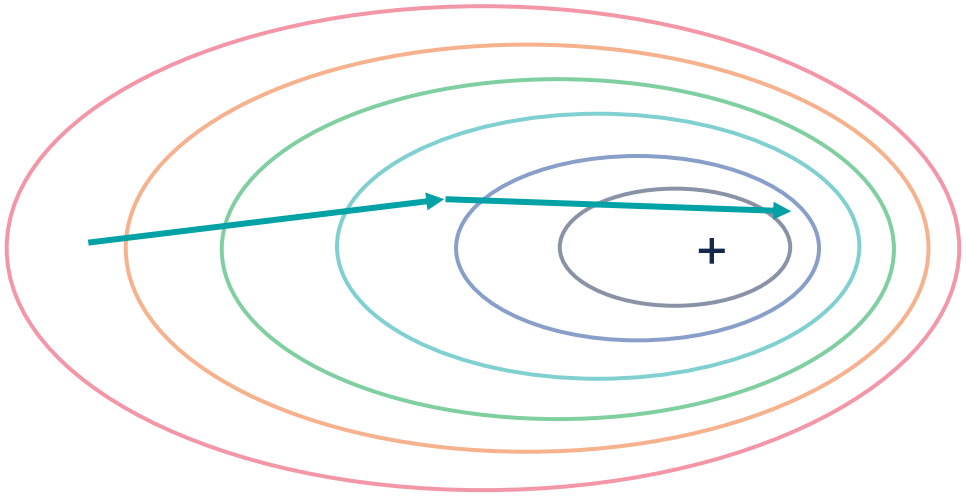
Deep neural network



Optimizers



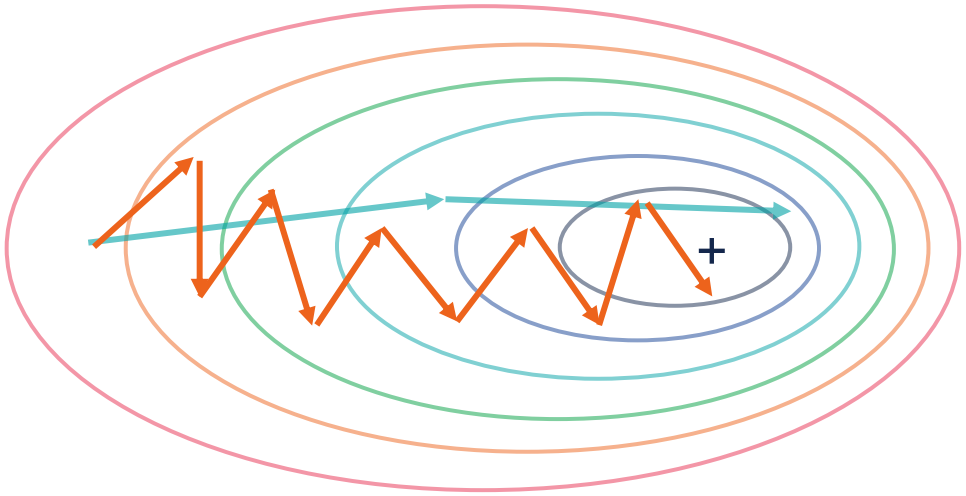
Optimizers



Gradient Descent

- Compute gradient and update weights after processing all samples
- Slow, not memory-efficient

Optimizers



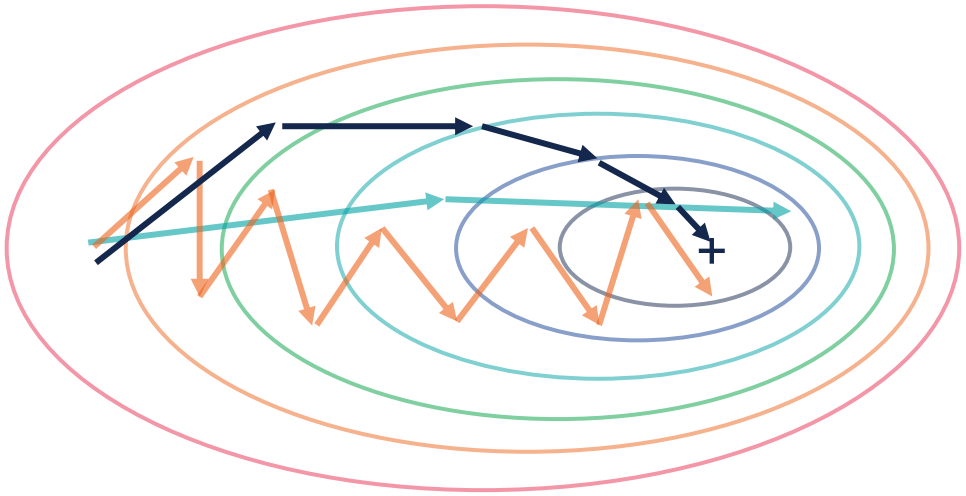
Stochastic Gradient Descent

- Compute gradient and update weights after each sample
- Noisy, compute-intensive

Stochastic Gradient Descent

- Average gradient over a batch of samples
- Faster updates, zig-zagging

Optimizers



Stochastic Gradient Descent

- Compute gradient and update weights for each sample
- Noisy, compute-intensive

Mini-batch Gradient Descent

- Average gradient over a batch of samples

ADAM

- Adaptive learning rates
- Computationally efficient
- Most popular optimizer

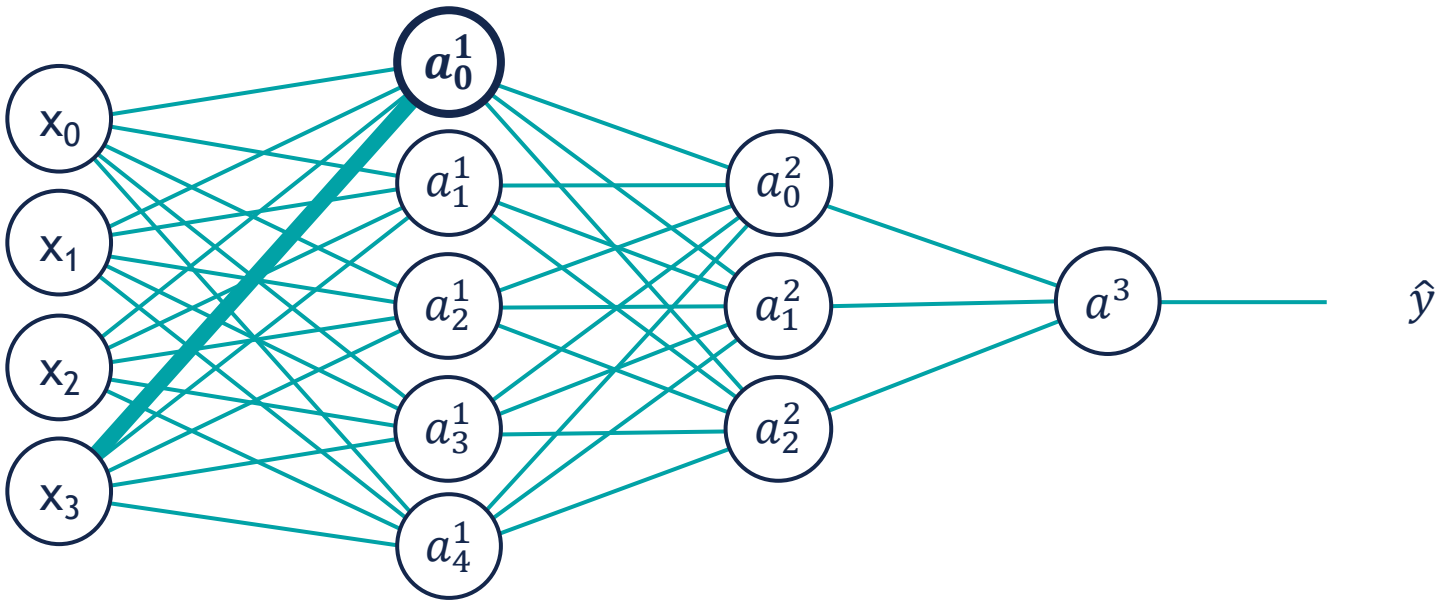
Vanishing/exploding gradients

Optimizers

Regularization

GPUs = matrix operations on fire

Regularization

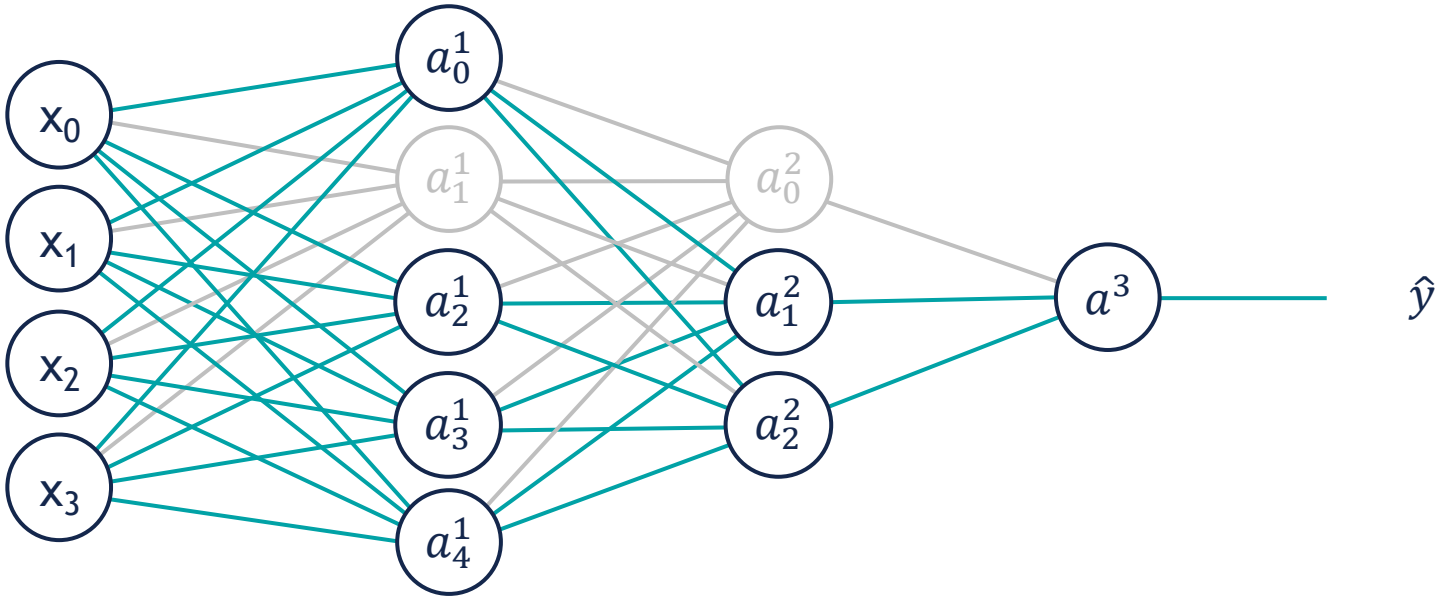


Batch normalization

- Large weights can cascade through the network and cause imbalances
- Normalize the output of each layer

cfr. feature normalization; features are learned in the network

Regularization



Dropout

- Neurons can become codependent, which contributes to overfitting
- For each training iteration, ignore a random fraction (p) of nodes
- *Requires more training iterations*

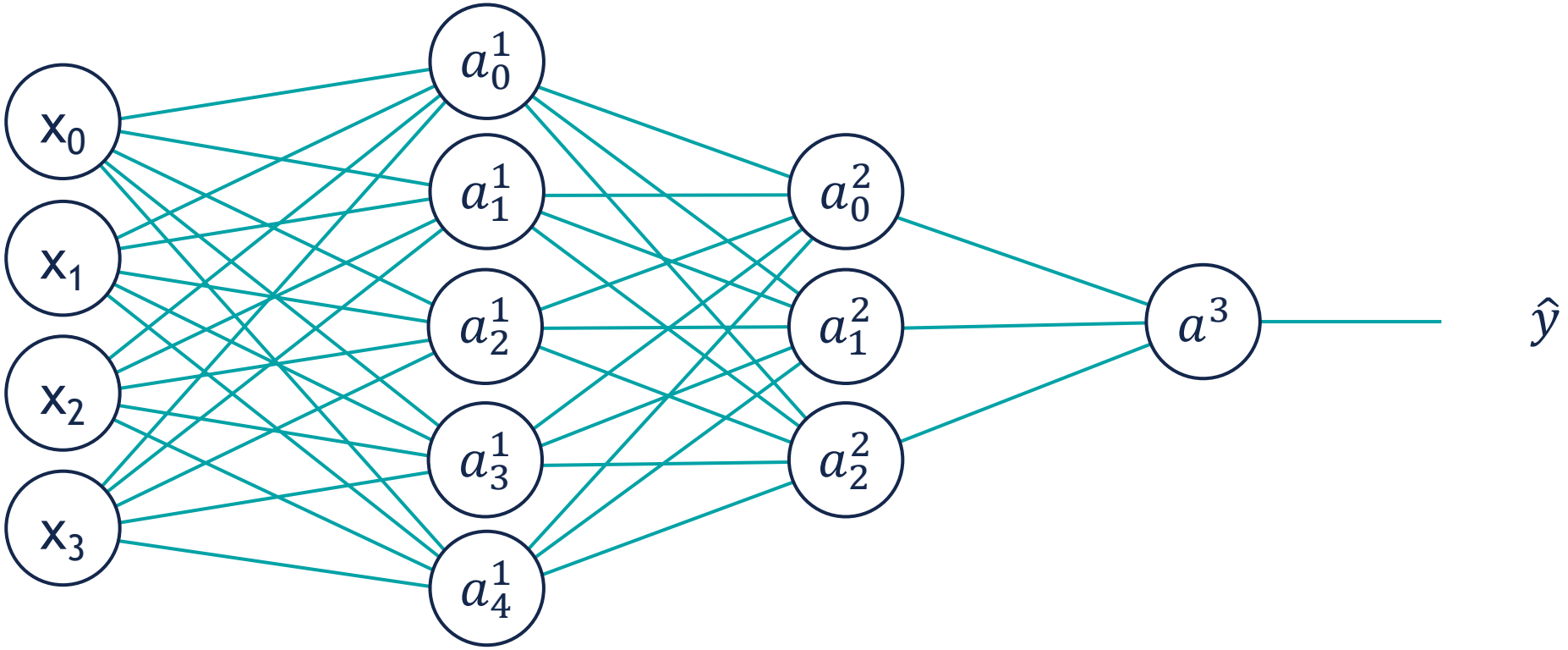
Vanishing/exploding gradients

Optimizers

Regularization

GPUs = matrix operations on fire

Neural network = matrix operations

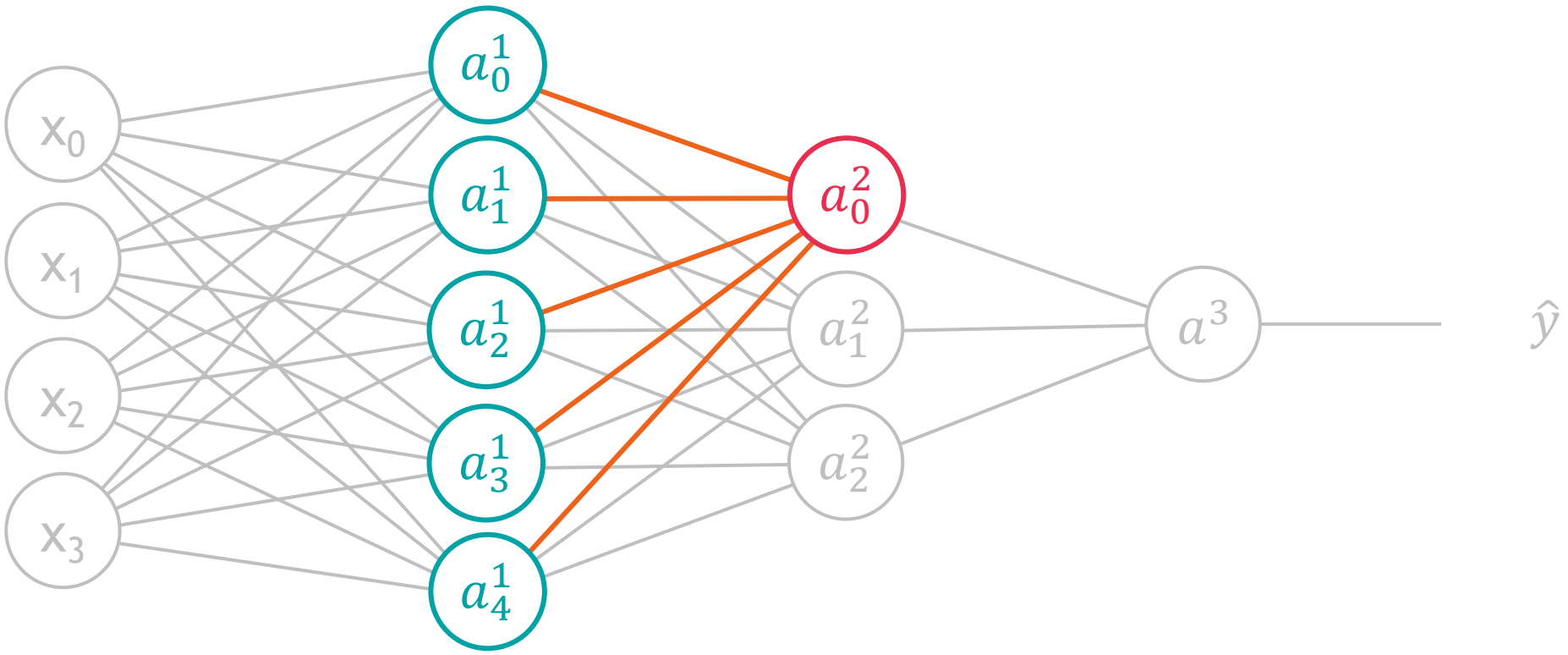


Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

Neural network = matrix operations

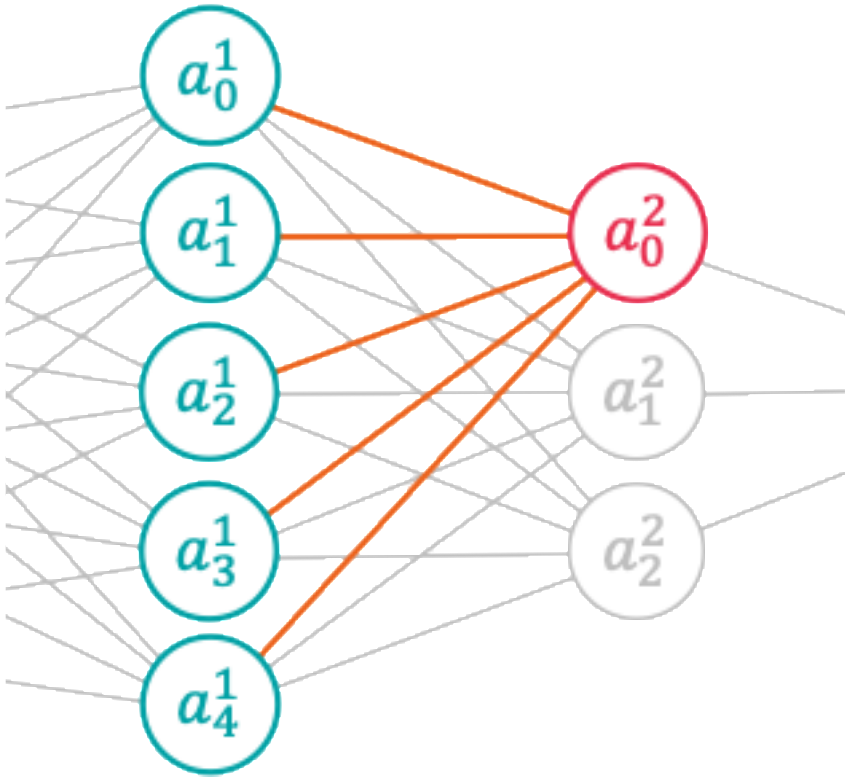


Input
(4 features)

Hidden layers
(5 and 3 nodes)

Output layer

Neural network = matrix operations



$$a_0^2 = \sigma(w_{00}a_0^1 + w_{01}a_1^1 + w_{02}a_2^1 + w_{03}a_3^1 + w_{04}a_4^1) + b_0$$

$$a_1^2 = \sigma(w_{10}a_0^1 + w_{11}a_1^1 + w_{12}a_2^1 + w_{13}a_3^1 + w_{14}a_4^1) + b_1$$

$$a_2^2 = \sigma(w_{20}a_0^1 + w_{21}a_1^1 + w_{22}a_2^1 + w_{23}a_3^1 + w_{24}a_4^1) + b_2$$

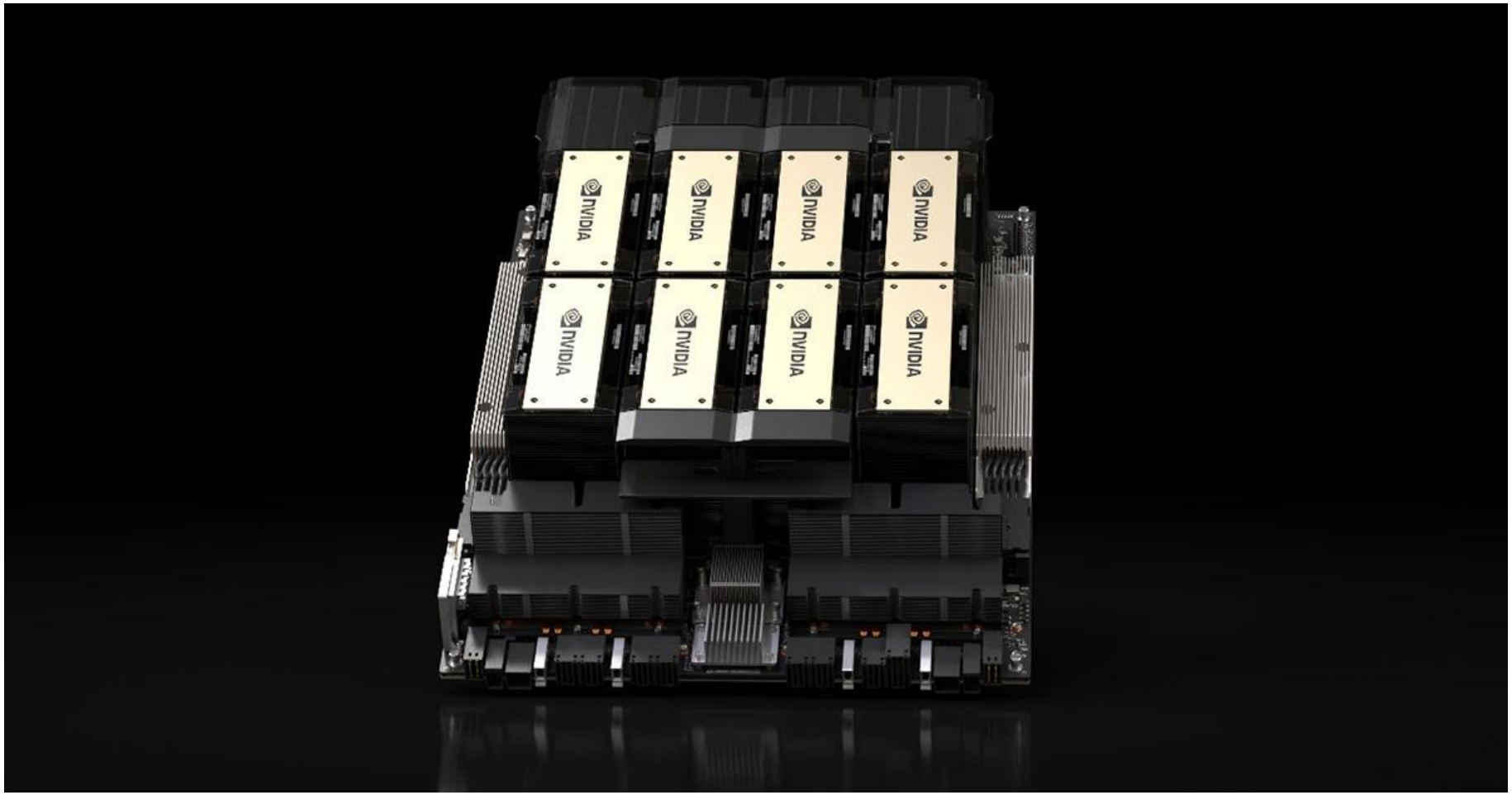
$$\downarrow$$

$$\begin{bmatrix} a_0^2 \\ a_1^2 \\ a_2^2 \end{bmatrix} = \begin{bmatrix} w_{00} & w_{01} & w_{02} & w_{03} & w_{04} \\ w_{10} & w_{11} & w_{12} & w_{13} & w_{14} \\ w_{20} & w_{21} & w_{22} & w_{23} & w_{24} \end{bmatrix} \cdot \begin{bmatrix} a_0^1 \\ a_1^1 \\ a_2^1 \\ a_3^1 \\ a_4^1 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

$$\downarrow$$

$$a^2 = \sigma(W^2 \cdot a^1 + \mathrm{b}^2)$$

GPUs = matrix operations on 🔥



GPUs = matrix operations on 🔥

EcoLogits

Estimate and track the environmental footprint of GenAI models at inference.

Made with 💚 by GenAI Impact non-profit.

https://ecologits.ai/latest/

VIB

GPUs = matrix operations on 🔥

Mistral AI's Large 2 Model
Lifecycle Analysis

		GHG Emissions	Water Consumption	Materials Consumption
INFRASTRUCTURE	1 **Model conception** Download and storage of training data, developers' laptops embodied impacts and power consumption	<1%	<1%	<1%
INFRASTRUCTURE	2 **Datacenter construction** Building and support equipment manufacturing	<1%	<1%	1.5%
INFRASTRUCTURE	3 **Hardware embodied impacts** Server manufacturing transportation and end-of-life	11%	5%	61%
COMPUTING	4 **Model training & inference** Power and water use of servers and support equipment	85.5%	91%	29%
COMPUTING	5 **Network traffic of tokens** Transfer of requests to inference clusters and responses back to users	<1%	<1%	<1%
USAGE	6 **End-user equipment** Embodied impacts and power consumption	3%	2%	7%
USAGE	7 **Downstream 'enabled' impacts** Indirect impacts that result from the product's use	N/A	N/A	N/A

- Hardware location
  - main factors of the environmental impact
  - *how is the energy produced?*

  → use local models (Mistral)
- Model size = impact

  → choose specialized mini-models
- Inference costs energy

  → use precise prompts

  → ask for short, grouped answers

VIB